

Aprendizaje por refuerzo en módulo lunar

Nicolás Gomez Claraco

Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla

Sevilla, España

nicgomcla@alum.us.es

nicogomezclaraco@gmail.com

Manuel Jesús Niza Cobo

Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla

Sevilla, España

mannizcob@alum.us.es

manolo3399@gmail.com

Resumen—El objetivo principal de este trabajo ha sido poner en práctica nuestros conocimientos en la asignatura acerca de aprendizaje por refuerzo, en concreto utilizando la técnica de Deep Q-Network (DQN) que combina Q-Learning con redes neuronales profundas para aprender una política que maximice la recompensa acumulada. El objetivo es aterrizar un módulo lunar de manera controlada en una zona de aterrizaje.

Se ha diseñado una arquitectura basada en red neuronal feed-forward con dos capas ocultas y función de activación ReLU, un buffer de repetición de experiencias y, además, se ha empleado una política ϵ -greedy para balancear la exploración y explotación durante el entrenamiento.

Al realizar este trabajo, se ha observado que el agente DQN es capaz de aprender gracias a la red neuronal y aterrizar correctamente el módulo lunar (LunarLander) en una proporción significativa de los episodios.

Palabras clave—Inteligencia Artificial, Aprendizaje por refuerzo, Deep Q-Network, ϵ -greedy, LunarLander, Redes Neuronales, TensorFlow.

I. INTRODUCCIÓN

Se nos planteó un proyecto en base a nuestros conocimientos acerca de redes neuronales estudiadas en la asignatura. El objetivo principal es diseñar un algoritmo de aprendizaje por refuerzo que permita, dentro del entorno de **Lunar Lander** aterrizar en buenas condiciones, en al menos un 30% de los casos. Para ello, utilizamos las librerías sugeridas: *Gymnasium* [1], *Keras* [2] y *TensorFlow* [3].

Para realizar ese algoritmo se planteó usar una técnica de **Deep Q-Network (DQN)**, que a diferencia del **Q-Learning clásico** donde se tiene una tabla donde cada par estado-acción tiene asociada una estimación de su valor Q, en DQN utiliza una red neuronal profunda como función que predice el valor Q. Esto permite que en lugar de almacenar explícitamente los valores Q en una tabla, DQN aprende a predecirlos, permitiendo escalar a grandes espacios de estado y abordar problemas complejos y continuos (como el de **Lunar Lander**).

Además, para que el aprendizaje del agente sea más estable y eficaz, se almacena información de experiencias pasadas *Replay Buffer* para que el agente pueda aprender de ellas varias veces, y se usa una segunda red *Red objetivo* que ayuda a que el proceso de aprendizaje sea más suave y menos propenso a errores.

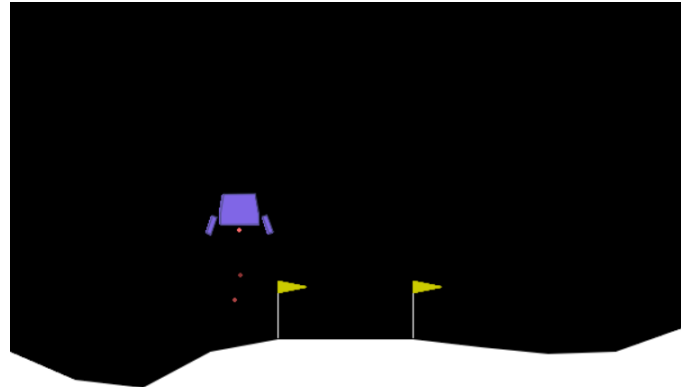


Fig. 1. Un ejemplo de entrenamiento de Lunar lander.

II. PRELIMINARES

En esta sección se hace una breve introducción a las técnicas empleadas y también a trabajos relacionados.

A. Algoritmos implementados

Se ha utilizado el algoritmo **Deep Q-Network (DQN)**, es una técnica de **Aprendizaje por Refuerzo** que usa redes neuronales profundas para aproximar la función de valor Q.

Las técnicas principales empleadas son:

- **Deep Q-Network:** Se emplea una red neuronal profunda con dos capas ocultas y activación *ReLU* para aproximar la función Q. Se usa para encontrar la política óptima en el algoritmo de aprendizaje por refuerzo.
- **Replay Buffer:** Se usa memoria de tipo *deque* que es una cola de doble extremo (se pueden sacar elementos de ambos extremos) para el buffer. Con esta memoria se almacenan experiencias realizadas anteriormente, el agente toma muestras aleatorias de esta memoria para actualizar su red neuronal y entrenar el modelo.
- **Target Network:** Se usa una red secundaria u objetivo que se actualiza de forma periódica copiando los parámetros de la red principal. Se usa para que el aprendizaje sea más estable, ya que evita que la red principal se actualice con objetivos que cambian continuamente.

- **Política ϵ -greedy:** Es una estrategia de exploración que selecciona acciones aleatorias con probabilidad ϵ y la mejor acción con probabilidad $1 - \epsilon$. Al principio del entrenamiento ϵ es alto para fomentar la exploración y el consecuente descubrimiento de nuevas estrategias, pero a medida que el agente va aprendiendo el valor de ϵ se reduce progresivamente, incitando a que el agente use los conocimientos adquiridos.

B. Trabajos relacionados

Se revisaron tres trabajos propuestos que han sido útiles para el desarrollo de los algoritmos implementados de **Deep Q-Networks (DQN)**.

- **Playing Atari with Deep Reinforcement Learning:** Se plantea un enfoque similar al elaborado en nuestro trabajo ya que usa **Deep Q-Network** (con los elementos que usamos como el *Replay Buffer*) para entrenar un juego de *Atari*, sin necesidad de conocimiento previo del juego [4].
- **Human-level control through deep reinforcement learning:** En el artículo se desarrolla una versión con mejoras en la arquitectura de la red y en la estabilidad del entrenamiento, además de resultados más robustos. Pone de ejemplo como las técnicas de aprendizaje profundo combinados con aprendizaje por refuerzo permite escalar a entornos complejos (como **Lunar Lander**) [5].
- **Deep Q-Networks Explained:** Proporciona una ayuda para entender el algoritmo de DQN. Explica de manera útil e intuitiva cómo funciona el *Replay Buffer*, la *Target Network* y la política ϵ -greedy [6].

III. METODOLOGÍA

La implementación se basa en el entorno *LunarLander-v2* de **OpenAI Gym**, el cual simula un módulo lunar que debe aterrizar entre dos banderas, evitando colisiones.

A. Arquitectura de la Red Neuronal (DQN)

Para entrenar al agente se ha empleado un algoritmo realizado que se basa en **Deep Q-Network (DQN)**, que utiliza una red neuronal profunda a través de *Keras* [2] y *TensorFlow* [3], definiendo los parámetros, capas ocultas y pesos.

La red neuronal definida tiene la siguiente estructura:

- Dos capas ocultas densas con 128 neuronas cada una, con pesos aleatorios al inicio y con función de activación *ReLU*:

$$f(x) = \max(0, x)$$

Esta función permite introducir no linealidad y es la recomendada para redes neuronales.

- Una capa de salida con tamaño igual al número de acciones posibles (4 acciones discretas), que devuelve los valores Q para cada acción con activación lineal.

La red neuronal recibe como entrada el estado actual (un vector de 8 variables continuas que son las variables del entorno **Lunar Lander**).

B. Replay Buffer (Experiencias)

El agente almacena experiencias en un buffer de tipo deque. Y permite guardar tuplas de la forma:

$$(s_t, a_t, r_t, s_{t+1}, done)$$

donde

- s_t es el estado actual,
- a_t es la acción tomada,
- r_t es la recompensa recibida,
- s_{t+1} es el siguiente estado,
- $done$ indica si el episodio ha terminado.

Este buffer permite almacenar experiencias recientes realizadas por el agente y entrenar la red gracias a conjuntos aleatorios de experiencias.

C. Política ϵ -greedy (Acciones)

En el proceso de entrenamiento se itera cada episodio donde el agente ejecuta diferentes acciones en el entorno siguiendo la política ϵ -greedy, que se define como:

$$a_t = \begin{cases} \text{acción aleatoria,} & \text{con probabilidad } \epsilon \\ \arg \max_a Q(s_t, a), & \text{con probabilidad } 1 - \epsilon \end{cases}$$

Donde ϵ es la probabilidad de exploración, que decae con el tiempo para favorecer la explotación del conocimiento aprendido:

$$\epsilon = \epsilon_{\min} + (1 - \epsilon_{\min})e^{-\frac{\text{episodio}}{500}}$$

Fig. 2.

realizar_accion()

Entrada: estado actual del entorno

Salida: $s', r, done, a$

```

1   $s \leftarrow estado\_actual()$ 
2  si  $rand() \leq \epsilon$  entonces
3       $a \leftarrow accion\_aleatoria()$ 
4  si no entonces
5       $Q \leftarrow q\_network(s)$ 
6       $a \leftarrow \arg \max Q$ 
7   $s', r, done \leftarrow tomar\_accion(a)$ 
8  retornar  $s', r, done, a$ 
```

Fig. 2. Algoritmo de realizar acción

D. Actualización del Modelo

El entrenamiento se realiza mediante un proceso iterativo en episodios, donde el agente interactúa con el entorno, almacena experiencias y actualiza la red neuronal.

La actualización de los pesos de la red se basa en la minimización de la **pérdida de Bellman**, definida como:

$$L(\theta) = E_{(s,a,r,s',done) \sim D} \left[(y - Q(s, a; \theta))^2 \right]$$

donde el objetivo y se calcula usando la red objetivo *Target Network* para mejorar la estabilidad:

$$y = r + \gamma \max_{a'} Q_{\text{target}}(s', a'; \theta^-)(1 - done)$$

Aquí,

- γ es el factor de descuento que pondera la importancia de recompensas futuras,
- θ son los parámetros actuales de la red principal,
- θ^- son los parámetros de la red objetivo, que se actualizan periódicamente copiando los pesos de la red principal.

Para calcular la pérdida entre los valores Q actuales y los objetivos se usa la **función Huber** [7] (que es menos sensible a valores atípicos).

Y finalmente se realiza una optimización de los gradientes donde se recortan y se actualizan los pesos de la red con un **optimizador Adam** [8], que ajusta manualmente la tasa de aprendizaje para cada parámetro, ideal para redes neuronales.

Fig. 3.

actualizar_modelo(agente)

Entrada: Agente con memoria y redes Q , Q^-

Salida: Pérdida l

```

1  si len( $\mathcal{M}$ ) < max( $B$ ,  $W$ ) entonces
2      retornar 0
3   $s, a, r, s', done \leftarrow ejemplo(\mathcal{M}, B)$ 
4   $q' \leftarrow Q^-(s')$ 
5   $a' \leftarrow argmax(Q(s'), axis = 1)$ 
6  batch_idx  $\leftarrow [0, \dots, B - 1]$ 
7   $q^* \leftarrow q'[batch\_idx, a']$ 
8   $y \leftarrow r + \gamma \cdot q^* \cdot (1 - done)$ 
9  Iniciar cálculo de gradientes
10      $q \leftarrow Q(s)$ 
11      $q_a \leftarrow q[batch\_idx, a]$ 
12      $l \leftarrow HuberLoss(y, q_a)$ 
13 Finalizar cálculo de gradientes
14  $\nabla l \leftarrow grad(l, Q.\theta)$ 
15  $\nabla l \leftarrow clip(\nabla l, 1.0)$ 
16 aplicar_optimizacion( $\nabla l, Q.\theta$ )
17 retornar  $l$ 
```

Fig. 3. Algoritmo de actualización del modelo

E. Actualización de la Red Objetivo

Para evitar oscilaciones durante el entrenamiento, tenemos una red objetivo separada que se actualiza cada cierto número de pasos (target_network_update_freq) copiando los pesos de la red principal:

$$\theta^- \leftarrow \theta$$

Esta técnica mejora la estabilidad del aprendizaje y ayuda a que los objetivos cambien lentamente. Fig. 4.

actualizar_red_objetivo(agente)

Entrada: Agente con redes Q y Q^-

Efecto: Copia los pesos de Q a Q^-

```

1  para cada ( $\theta^-, \theta$ ) en zip( $Q^-, Q$ ) hacer
2       $\theta^- \leftarrow \theta$ 
```

Fig. 4. Algoritmo de actualización de la red objetivo

Un ejemplo de pseudocódigo del método de entrenamiento: Fig. 5.

entrenamiento(agente)

Entrada: Un agente predefinido con entorno LunarLander

Salida: Agente entrenado y estadísticas de entrenamiento

```

1  inicializar lista de recompensas R
2  inicializar lista de pérdidas L
3   $R^* \leftarrow -\infty$ 
4  registrar_tiempo_inicio
5  Para cada episodio en  $[1, \dots, episodios]$  hacer
6       $s \leftarrow reset(agente.lunar)$ 
7      total_recompensa  $\leftarrow 0$ 
8      perdida_episodio  $\leftarrow 0$ 
9      pasos_episodio  $\leftarrow 0$ 
10     fin  $\leftarrow$  falso
11     mientras no fin hacer
12          $s', r, done, a \leftarrow realizar_accion()$ 
13         total_recompensa  $\leftarrow total\_recompensa + r$ 
14         guardar_memoria( $s, a, r, s', done$ )
15         estado_modulo  $\leftarrow s'$ 
16          $s \leftarrow s'$ 
17         pasos_episodio  $\leftarrow pasos\_episodio + 1$ 
18          $C \leftarrow C + 1$ 
19         si  $C > W \wedge C \bmod 4 = 0$  entonces
20              $l \leftarrow actualizar\_modelo(agente)$ 
21             perdida_episodio  $\leftarrow perdida\_episodio + l$ 
22         si episodio mod  $U = 0$  entonces
23             actualizar_red_objetivo(agente)
24         si  $\epsilon > \epsilon_{min}$  entonces
25              $\epsilon \leftarrow \epsilon_{min} + (1 - \epsilon_{min}) \cdot e^{-e/500}$ 
26          $R.añadir(total\_recompensa)$ 
27          $L.añadir(perdida\_episodio/pasos\_episodio)$ 
28          $\bar{R} \leftarrow media(R[-100 :])$ 
29         si  $\bar{R} > R^*$  entonces
30              $R^* \leftarrow \bar{R}$ 
31     cada 100 episodios: imprimir estadísticas
32 imprimir Entrenamiento finalizado
33 guardar_modelo(agente)
```

Fig. 5. Algoritmo de entrenamiento DQN Agent

IV. RESULTADOS

En esta sección se detallan los experimentos realizados y los resultados obtenidos tras entrenar un agente **Deep Q-Network (DQN)** para resolver el entorno **Lunar Lander**. Además, se analizan los resultados obtenidos, destacando los aspectos más relevantes del comportamiento del agente.

A. Modelo Inicial

1) *Configuración experimental:* El agente ha sido entrenado sobre el entorno *LunarLander-v2* del paquete **Gymnasium** [1], a través de un entorno propio *LunarLanderEnv*. La configuración empleada se basa en los siguientes hiperparámetros:

- **Factor de descuento γ :** 0.99
- **Tasa de exploración inicial ϵ :** 1.0
- **Tasa de decaimiento de ϵ :** 0.998
- **Valor mínimo de ϵ :** 0.01
- **Tasa de aprendizaje:** 0.001
- **Tamaño del batch:** 64
- **Tamaño de la memoria de replay:** 10 000 experiencias.
- **Frecuencia de actualización de la red objetivo:** cada 400 episodios.
- **Warm-up:** 10 000 pasos antes de comenzar a entrenar la red.
- **Número de episodios de entrenamiento:** 5 000 episodios.

El objetivo del experimento era determinar si un agente DQN con una red neuronal de arquitectura sencilla (dos capas ocultas de 128 neuronas cada una) era capaz de aprender a resolver el entorno de **Lunar Lander**, aumentando considerablemente el número de episodios.

2) *Resultados obtenidos:* A continuación, se resumen los resultados obtenidos durante los experimentos:

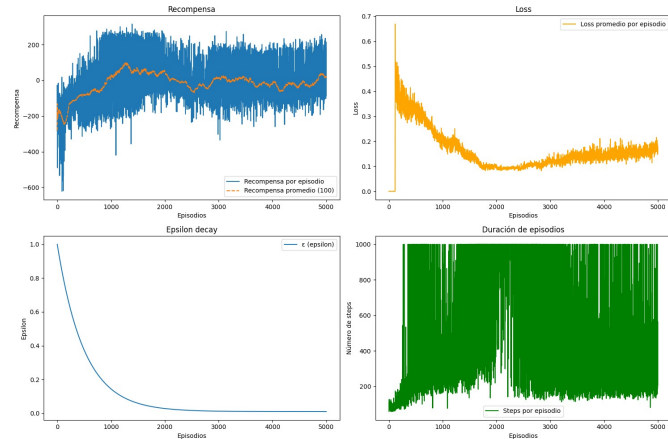


Fig. 6. Evolución de las recompensas durante el entrenamiento

- Durante los primeros 1,500 episodios, el agente presentó un comportamiento predominantemente aleatorio debido a un alto valor de ϵ , favoreciendo así la exploración del entorno.

- A partir del episodio 500, se comenzó a observar una mejora progresiva en las recompensas promedio, indicando que el agente estaba empezando a aprender una política más efectiva. De forma paralela, la duración de los episodios también aumentó progresivamente, reflejando estrategias que permitían mantener el control del vehículo durante más tiempo y evitar situaciones catastróficas.
- La mejor recompensa promedio alcanzada fue de aproximadamente 50, lograda en torno al episodio 2,200.
- La evolución del valor de ϵ siguió la estrategia exponencial prevista, descendiendo gradualmente y estabilizándose en torno a su valor mínimo de 0.01 a partir del episodio 2,000.
- La función de pérdida mostró un descenso sostenido durante los primeros 2,000 episodios, indicando un aprendizaje efectivo, pero a partir del episodio 2,000, la función de pérdida comenzó a incrementarse nuevamente, sugiriendo una degradación en la estabilidad del modelo y dificultades en la convergencia.
- Entre los episodios 2,000 y 3,000, el agente pareció entrar en una fase de estancamiento, donde el agente incrementaba la duración de los episodios para sobrevivir empeorando las recompensas promedio.
- Desde el episodio 3,000 hasta el final del entrenamiento, el modelo mostró incapacidad para converger correctamente, alternando entre estrategias ineficientes que no mejoraban significativamente las recompensas y que producían una elevada variabilidad en la duración de los episodios.

3) *Análisis de los resultados:* Los resultados muestran que el agente DQN no fue capaz de converger a una política competente para el entorno **Lunar Lander**:

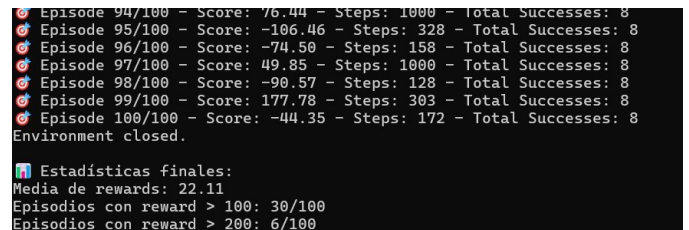


Fig. 7. Test de las recompensas del modelo

El modelo ha conseguido una puntuación promedio realmente baja. En una prueba constituyente de 100 episodios solo ha conseguido un 30% aterrizajes mediocres y tan solo un 6% donde se ha logrado un aterrizaje excepcional. Algunos de los motivos a los que podemos atribuir este fracaso son:

- El uso de la estrategia ϵ -greedy permitió un equilibrio adecuado entre exploración y explotación durante el proceso de aprendizaje aunque no fue suficiente para conseguir una política óptima.
- El uso de un número de episodios con tamaño suficiente de 5000, y la técnica de *Target Network* con una ac-

tualización de 400 episodios no consiguieron lograr la estabilidad del entrenamiento y la tasa de aprendizaje con un valor de 0.001 fue inapropiada para el modelo ocasionando oscilaciones bruscas en la política aprendida.

En resumen, el agente fracasó en su tarea de aprender una política efectiva y estable tras aproximadamente 5,000 episodios de entrenamiento. Revisaremos la arquitectura de la red y, sobre todo, los hiperparámetros para poder conseguir un modelo capaz de resolver el entorno.

B. Modelo Final

1) *Configuración experimental:* El agente ha sido entrenado sobre el entorno *LunarLander-v2* del paquete **Gymnasium** [1], a través de un entorno propio *LunarLanderEnv*. La configuración empleada se basa en los siguientes hiperparámetros:

- **Factor de descuento γ :** 0.99.
- **Tasa de exploración inicial ϵ :** 1.0.
- **Tasa de decaimiento de ϵ :** 0.998.
- **Valor mínimo de ϵ :** 0.01.
- **Tasa de aprendizaje:** 0.0005.
- **Tamaño del batch:** 128.
- **Tamaño de la memoria de replay:** 100 000 experiencias.
- **Frecuencia de actualización de la red objetivo:** cada 400 episodios.
- **Warm-up:** 10 000 pasos antes de comenzar a entrenar la red.
- **Número de episodios de entrenamiento:** 2 000 episodios.

El objetivo de este segundo modelo era determinar si un agente DQN con una red neuronal de arquitectura sencilla similar a la red anterior (dos capas ocultas de 128 neuronas cada una) pero con cambios significativos en sus hiperparámetros, sería capaz, en un menor número de episodios, de superar al modelo anterior.

2) *Resultados obtenidos:* A continuación, se resumen los resultados obtenidos durante los experimentos:

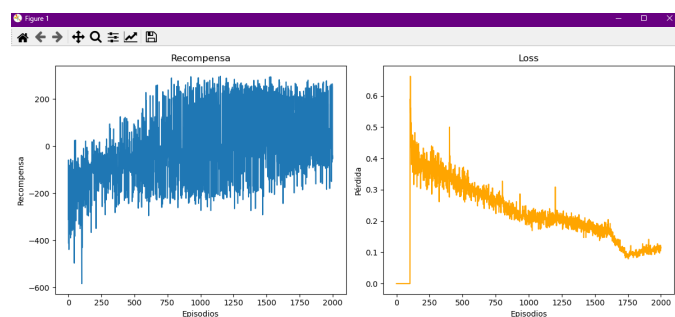


Fig. 8. Evolución de las recompensas durante el entrenamiento

Lo primero es aclarar que no se ha apreciado ninguna diferencia en las gráficas de epsilon decay ni en las de duración del episodio, por lo que hemos estimado conveniente centrarnos en las otras dos.

- Durante los primeros 500 episodios, el agente volvió a mostrar un comportamiento predominantemente aleatorio debido al alto valor de ϵ .
- Entre el episodio 500 y el 800, se experimentó el periodo de mayor mejora progresiva en las recompensas promedio, indicando que el agente había empezado a aprender una política capaz de alcanzar mejores puntuaciones.
- A partir del episodio 1 000, el agente no fue capaz de conseguir mejores picos de recompensas esto se debe a que sencillamente no se puede, es imposible alcanzar puntuaciones mucho mejores que las máximas obtenidas (273).
- Desde el episodio 1 000 hasta el final del entrenamiento en el episodio 2 000 podemos apreciar que el modelo va reduciendo de forma consistente el número de aterrizajes catastróficos, suavizando la curva por la parte inferior de la gráfica y mejorando el promedio de las recompensas de esa manera.
- La función de pérdida loss, mostró un comportamiento decreciente a medida que avanzaba el entrenamiento, estabilizándose en valores bajos, lo que en esta ocasión si nos permite garantizar una correcta convergencia del modelo.

3) *Análisis de los resultados:* Los resultados muestran que esta nueva versión del agente DQN fue capaz de aprender una política competente para el entorno **Lunar Lander**:

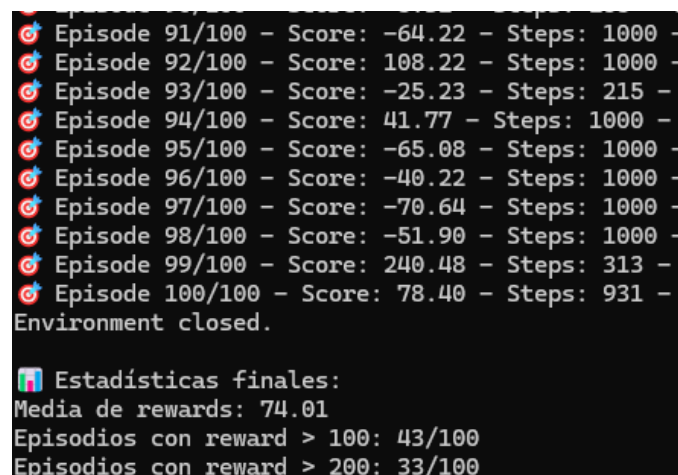


Fig. 9. Test de las recompensas del modelo

El modelo ha conseguido una puntuación promedio nada despreciable, consiguiendo una tasa de aterrizajes exitosos de un 33%; algunos motivos a los que podemos atribuir esta mejora son:

- El Replay Buffer ampliado, diez veces el tamaño inicial, demostró ser un componente clave en el éxito del modelo, proporcionando estabilidad al aprendizaje al permitir el reuso eficiente de experiencias pasadas y estabilizar la distribución de datos.

- La reducción de la tasa de aprendizaje ha permitido suavizar notablemente el proceso de convergencia en las fases finales del entrenamiento.
- El aumento del tamaño del batch de 64 a 128 unidades demostró mejoras significativas en la estabilidad del entrenamiento, reduciendo la varianza de las actualizaciones de gradiente y mejorando el aprovechamiento del recién ampliado replay buffer.

En resumen, el agente logró aprender una política efectiva y estable tras aproximadamente 2000 episodios de entrenamiento. El comportamiento final del agente permitía aterrizajes exitosos en más de un 30% de los episodios, cumpliendo así con el criterio de resolución.

V. CONCLUSIONES

En este proyecto hemos desarrollado un agente que aprende a controlar un módulo lunar en el entorno, utilizando un algoritmo basado en **Deep Q-Network (DQN)**. Para ello, hemos empleado *Replay Buffer*, una red principal *Q-Network*, una red objetivo *Target Network*, y una política de exploración/explotación ϵ -greedy entre algunas de las herramientas importantes.

Durante el entrenamiento, hemos podido comprobar que el agente es capaz de aprender una estrategia resolutoria basada en los conjuntos de experiencias ya almacenados en el buffer. A medida que avanza el entrenamiento, el agente pasa de explorar acciones aleatorias a realizar acciones basadas en la experiencia, lo que le permite conseguir aterrizajes más suaves y exitosos entre las dos banderillas. Gracias a esto, hemos llegado a una tasa de resolución superior al 30%, considerando como aceptables los resultados con recompensas superiores a 200, como indica la web oficial de *gymnasium* [1], lo cual indica que el objetivo inicial del problema se ha cumplido.

El comportamiento del agente se ha desarrollado de manera estable, lo que confirma que las herramientas desarrolladas y los hiperparámetros establecidos han sido integrados de manera exitosa y funcionan bien en su conjunto.

Aún siendo exitosa esta versión básica del programa implementado, se pueden implementar otras mejoras que mejoren tanto la eficiencia como la efectividad de los algoritmos:

- Explorar arquitecturas de red más profundas o el uso de técnicas avanzadas como *Dueling DQN*, *Double DQN* o *Prioritized Experience Replay*, que permite aprender de las experiencias más útiles en vez de un conjunto aleatorio.
- Automatizar la búsqueda de los mejores hiperparámetros (tamaño de la red, tasa de aprendizaje, tasa de decaimiento de epsilon, etc.) para sacar el máximo rendimiento de estas variables.
- Realizar más pruebas de entrenamiento y mejorar la calidad de las mismas para tener en cuenta parámetros menos sensibles para la puntuación pero que afectan a la calidad del vuelo como la velocidad angular en el

aterrizaje, esto puede ayudar a mejorar la comprensión de los resultados obtenidos.

En conclusión, el trabajo nos ha ayudado a comprender un poco más cómo funciona el aprendizaje por refuerzo, y sobre todo en este caso en concreto de **Deep Q-Network**. Hemos podido comprender de manera simple cómo elaborar un algoritmo para que un entorno del tipo **Lunar Lander** aprenda a aterrizar de manera autónoma gracias, en gran parte, al apoyo visual que se nos ha proporcionado y que nos ha ayudado a comprender mejor las estrategias elegidas por el modelo. Además, el proceso de búsqueda del mejor modelo fue bastante entretenido. Aún queda mucho margen para seguir probando mejoras y aprendiendo acerca de este campo de la inteligencia artificial.

REFERENCIAS

- [1] F. Foundation, "Lunar lander — gymnasium documentation," 2025. [Online]. Available: https://gymnasium.farama.org/environments/box2d/lunar_lander/
- [2] F. Chollet and contributors, "Keras documentation," 2025. [Online]. Available: <https://keras.io/>
- [3] T. Developers, "Tensorflow documentation," 2025. [Online]. Available: <https://www.tensorflow.org/>
- [4] V. Mnih *et al.*, "Playing atari with deep reinforcement learning," 2013, arXiv preprint arXiv:1312.5602. [Online]. Available: <https://www.cs.toronto.edu/~vmnih/docs/dqn.pdf>
- [5] —, "Human-level control through deep reinforcement learning," 2015, *nature*, vol. 518, pp. 529–533, Feb. 2015.
- [6] D. M. Ziegler, "Deep q-networks explained," 2016, *lessWrong*. [Online]. Available: <https://www.lesswrong.com/posts/kyvCN9oAwJCuevo/deep-q-networks-explained>
- [7] P. J. Huber, "Robust estimation of a location parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964. [Online]. Available: <https://projecteuclid.org/euclid.aoms/1177703732>
- [8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014. [Online]. Available: <https://arxiv.org/abs/1412.6980>